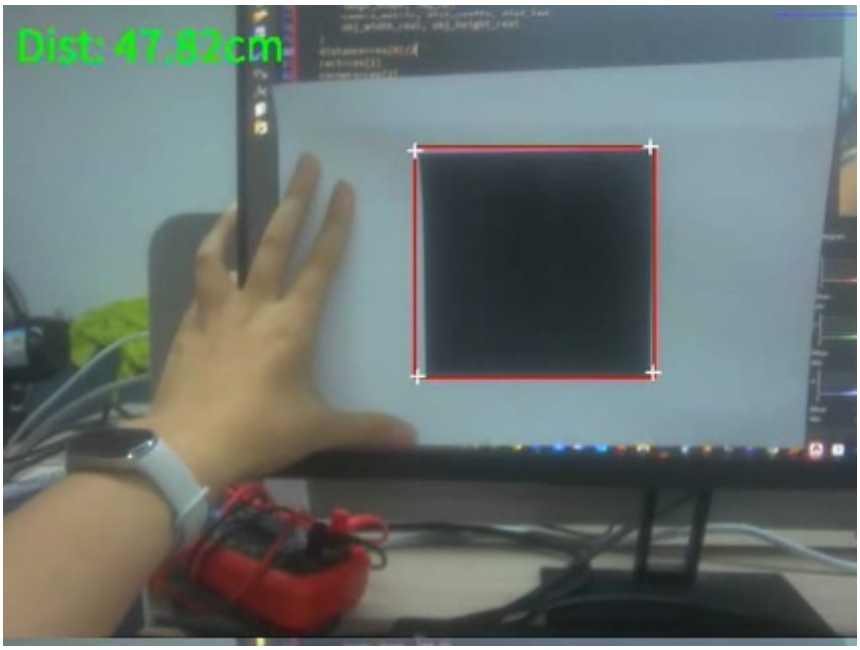
寻找矩形并测距

寻找矩形并测距
- 例程效果
- 原理讲解
  - 什么是图像轮廓检测与PnP距离估计
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置相机内参和畸变系数
    - 设置帧率计时器
    - 图像处理与轮廓检测及距离估计
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/27.rgb888_pnp_distance_from_corners.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现图像轮廓检测，并使用 PnP 算法对检测到的轮廓进行距离估计。



原理讲解

什么是图像轮廓检测与PnP距离估计

图像轮廓检测是一种计算机视觉技术，用于提取图像中物体的边界或边缘轮廓。PnP距离估计则基于透视投影原理，结合相机参数和物体实际尺寸，计算物体在三维空间中的距离。整个过程常用于实时物体定位和深度估计。

- **操作原理**：

    - **轮廓检测**：首先通过边缘检测（如Canny）或阈值分割提取图像中的轮廓，然后筛选出规则的形状（如矩形）。本代码中，轮廓检测可能内嵌在PnP函数中，处理图像以找到关键点（例如四个角点）。
    - **PnP距离估计**：PnP（Perspective-n-Point）算法使用已知的3D点（基于物体实际尺寸）和对应的2D图像点（检测到的轮廓角点），结合相机内参矩阵和畸变系数，通过最小化重投影误差计算物体的位姿和距离。公式涉及求解PnP问题，通常使用EPnP或DLT算法。
- **效果**：该操作可以检测图像中的轮廓，并在图像上绘制边界框、角点和距离文本，适用于需要精确定位的场景。检测结果受图像质量、灯光条件和参数影响，可能出现噪声或误差。
- **应用场景**：轮廓检测和PnP距离估计广泛应用于机器人导航（如避障）、增强现实（AR）、物体跟踪和工业检测（如测量产品尺寸）。在嵌入式设备上，这种方法结合了高效性和准确性，适合资源有限的环境。

在实际应用中，相机参数的准确性和物体实际尺寸的测量是关键，以确保距离估算的可靠性。

代码概述

导入模块

```
import time, os, gc
from machine import Pin
from media.sensor import *
from media.display import *
from media.media import *
import _thread
import cv_lite                  # 需要实现对应的 native C 接口
import ulab.numpy as np
```

设置图像尺寸

```
image_shape = [480, 640]
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2,width=1280,height=960,fps=90)
sensor.reset()
# 设置采集图片的分辨率
sensor.set_framesize(w=image_shape[1], h=image_shape[0],chn=CAM_CHN_ID_0)
sensor.set_pixformat(Sensor.RGB888)
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 90 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理轮廓检测）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=50)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置相机内参和畸变系数

> 此处给的摄像头内参为基于亚博智能K230上的GC2093摄像头标定后得到的数据

```
camera_matrix = [
    1199.1495245491617, 0.0, 670.4213222250077,
    0.0, 1200.6879329729338, 475.11122810359876,
    0.0, 0.0, 1.0
]
dist_coeffs = [-0.0331624688297151, 0.04145795463721313, 0.0016964134879770974,
0.0008834973490863845, -0.05414957483933673]
dist_len = len(dist_coeffs)

# --------------------------------
# 目标实际尺寸（单位 cm）
# --------------------------------
obj_width_real = 12.8
obj_height_real = 12.8
```

- `camera_matrix`：相机内参矩阵，用于PnP距离估算。
- `dist_coeffs`：相机畸变系数，用于校正镜头畸变。
- `obj_width_real` 和 `obj_height_real`：检测物体的实际物理尺寸（单位：厘米），用于计算距离。

设置帧率计时器

```
clock = time.clock()
```

初始化帧率计时器，用于计算和显示每秒处理的帧数（FPS）。

图像处理与轮廓检测及距离估计

```
while True:
    clock.tick()

    img = sensor.snapshot()
    img_np = img.to_numpy_ref()

    # 距离估计（通过轮廓+PnP）
    res = cv_lite.rgb888_pnp_distance_from_corners(
        image_shape, img_np,
        camera_matrix, dist_coeffs, dist_len,
        obj_width_real, obj_height_real
    )
    distance=res[0]/2
    rect=res[1]
    corners=res[2]
```

```
    # 如果距离估计成功
    if distance > 0:
        img.draw_string_advanced(10, 10, 32, "Dist: %.2fcm" % distance, color=
(0, 255, 0))
        img.draw_rectangle(rect[0], rect[1], rect[2], rect[3], color=(255, 0,
0), thickness=2)
        img.draw_cross(corners[0][0],corners[0][1],color=
(255,255,255),size=5,thickness=2)
        img.draw_cross(corners[1][0],corners[1][1],color=
(255,255,255),size=5,thickness=2)
        img.draw_cross(corners[2][0],corners[2][1],color=
(255,255,255),size=5,thickness=2)
        img.draw_cross(corners[3][0],corners[3][1],color=
(255,255,255),size=5,thickness=2)
    else:
        img.draw_string_advanced(10, 10, 32, "No Rect Found", color=(255, 0, 0))

    # 显示图像
    Display.show_image(img)

    print("contour_pnp:", clock.fps())
#    print("Distance:", distance)
    gc.collect()
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **轮廓检测和距离估计处理**：调用 `cv_lite.rgb888_pnp_distance_from_corners()` 函数执行轮廓检测和PnP距离估算，返回距离、矩形和角点信息。
- **绘制和显示**：在图像上绘制距离文本、矩形框和角点标记，并显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `camera_matrix` **和** `dist_coeffs`：相机内参和畸变系数，仅在相机变化时调整
- `obj_width_real` **和** `obj_height_real`：物体实际物理尺寸（单位：厘米），基于真实物体测量调整；准确值有助于减少距离计算误差，建议根据目标物体重新校准。
- **其他参数**：本代码中，轮廓检测参数（如阈值）可能内嵌在cv_lite函数中，无法直接调整。如果需要优化检测精度，建议通过测试图像调整cv_lite模块的内部设置或扩展参数。